🚀 Lecture 02.02: Variables & Memory Architecture in Python

📌 Executive Architectural Summary

In Python, variables are not static memory containers (like in C/C++ where a variable name maps to a fixed memory slot). Instead, variables in Python are **dynamic pointer references** (labels) that bind to heap-allocated objects.

🔬 Core Engineering Mechanics

1. Identifier Naming Rules

Identifiers are symbolic names used to identify variables, functions, classes, and modules:

1. **Valid Characters:** Must begin with an ASCII letter ($a - z$, $A - Z$) or an underscore (`_`). Subsequent characters can include digits ($0 - 9$).
2. **Invalid Initial Characters:** Cannot begin with a numeric digit (e.g., `2name` raises `SyntaxError`).
3. **Reserved Keywords:** Cannot use Python keywords (`for` , `class` , `def` , `if` , etc.).
4. **Case-Sensitivity:** Python is strictly case-sensitive. `age` , `Age` , and `AGE` represent three entirely distinct identifier bindings.

2. Pointer Binding Mechanics

```
Variable Name (Identifier) --------> [ Heap Object: PyObject ]
      "age"                          - ob_refcnt: 1
                                     - ob_type: int
                                     - ob_digit: 30
```

When executing `age = 30` :

- CPython allocates a `PyLongObject` with value `30` on the heap.
- The symbol `age` in the local namespace dictionary is bound to the memory address of that `PyLongObject` .

💻 Production Python Implementation

```
import sys

# 1. Variable Assignment & Re-binding
name = "Shradha"
age = 30
pi_constant = 3.14159

print(f"Name: {name} (Memory ID: {id(name)})")
print(f"Age: {age} (Memory ID: {id(age)})")
print(f"PI: {pi_constant} (Memory ID: {id(pi_constant)})")

# 2. Case-Sensitivity Verification
age = 25
Age = 40
AGE = 65
print(f"age: {age}, Age: {Age}, AGE: {AGE} (All 3 distinct!)")

# 3. Dynamic Re-binding (Variable points to a completely different type)
x = 100
print("x is int:", type(x), "at", id(x))
x = "Now I am a string"
print("x is str:", type(x), "at", id(x))
```